SR-MPLS Training Lab

M8 Capstone Break/Fix and Brief

Estimated time: 90 min

Objective

Restore a network carrying three simultaneous faults across the underlay, the SR transport, and the L3VPN service, then explain what happened. You will apply the eight-step troubleshooting flow end to end, working strictly from the lowest layer upward, prove that full customer reachability is back, and deliver a five-minute root-cause brief. This capstone tests diagnosis and communication, not your ability to type new configuration.

Before you start: M0 through M7 are complete and previously verified on this pod, and the instructor has loaded the capstone fault set so that service is currently down before you begin.

Concept primer

SR-MPLS is a strictly layered stack, and every layer depends on the one below it. IS-IS forms the underlay and runs SPF, which decides both the IGP next-hop and the set of node-SIDs each router can reach. Flex-Algo and the colored transport planes ride on that SPF result. The MPLS forwarding table and inet.3 tunnels are programmed from those SIDs, and only then does the BGP L3VPN service resolve its VPN routes onto the transport. Because each layer is computed from the one under it, a fault low in the stack does not announce itself politely at the top. It reshapes the path or the label table silently, so the service simply fails while the real cause sits two layers down.

That is why the discipline is underlay before transport before service. A broken metric moves the SPF path, which drags every SR label-switched path with it, so a transport fault on the newly chosen path can look like a healthy underlay hiding a service problem. A label clash on the active plane breaks MPLS forwarding even after the metric is corrected, so fixing the underlay alone is not enough. And a VRF import mismatch cannot be judged at all while the transport under it is broken, because the routes never had a tunnel to resolve onto. The eight steps in the troubleshooting guide are ordered exactly for this reason: confirm IS-IS adjacencies, then the algo and colored planes, then VRF steering, then CoS

mapping and EXP rewrite, then the IS-IS route and TI-LFA, then the mpls.0 transit labels, then the inet.3 tunnel resolution. You always fix the lowest broken layer first, then re-run the checks above it, because clearing the lower fault often changes what the higher checks report.

A good root-cause brief is short, ordered, and honest about method. It names each fault in plain terms, states the single command output that exposed it, and explains why the layer order mattered: which lower fault was masking which higher one, and why you could not have trusted the service-layer evidence until the transport under it was clean. It closes with the customer impact and the confirmation that reachability is restored. The goal is that a listener who never saw your terminal can repeat your reasoning and reach the same conclusion.

Build steps

1. **Step 1. Confirm the IS-IS underlay adjacencies are all Up.**

```
show isis adjacency
```

Note: On every router each expected neighbor must read State Up. PE1 and PE2 have two plane neighbors each; P and C routers have two or three. If any adjacency is Down or Init, stop and fix that first, because with a missing adjacency no SIDs are distributed and every check above is unreliable. Adjacencies Up does not clear the underlay: a wrong metric still shows every neighbor Up.

2. **Step 2. Check the Flex-Algo and colored transport planes in inetcolor.0.**

```
show route table inetcolor.0
```

Note: For each remote PE loopback expect two colored entries, one ending in -128 (IGP metric, algo 128) and one ending in -129 (delay, algo 129), each pushing the remote node-SID. From PE1 both 10.20.255.2-128<c>/64 and 10.20.255.2-129<c>/64 should be present. If a plane looks wrong here, suspect the SPF result below it before blaming Flex-Algo itself.

3. **Step 3. Check the VRF service steering. Defer judgement until the layers below are clean.**

```
show route table CUST_A.inet.0
show route receive-protocol bgp 10.20.255.11 table bgp.l3vpn.0
```

Note: A clean result is the remote CE prefixes present in CUST_A.inet.0. If the VRF table is empty while the routes are clearly present in bgp.l3vpn.0, that points at an import route-target mismatch. Do not trust this reading until the transport under it

is proven healthy; a broken tunnel produces the same empty table for a completely different reason.

4. **Step 4. Check the CoS interface mapping on transit links.**

```
show class-of-service interface ge-0/0/1
```

Note: The physical interface must show Scheduler map SR-MPLS-SMAP, and the .0 unit must list Rewrite-Output SR-MPLS-EXP-REWRITE and Classifier SR-MPLS-EXP-CLASS. A missing rewrite collapses QoS from that hop onward but does not itself drop reachability.

5. **Step 5. Check the EXP rewrite rule marks the right code points.**

```
show class-of-service rewrite-rule name SR-MPLS-EXP-REWRITE
```

Note: premium-priority at low loss must map to code point 101 and best-effort to 000. If premium-priority maps to 000, CUST_B traffic lands in the best-effort queue at every transit hop regardless of PE classification.

6. **Step 6. Check the IS-IS route to the remote SID and its TI-LFA backup.**

```
show isis route 10.20.255.2 extensive
show isis database
```

Note: From PE1 the route to PE2 should show the expected primary paths toward SID 16002 and TILFA computed MPLS paths with Valid: 1. This is where an underlay metric fault becomes visible: compare the advertised link metrics in the database against the design and check the total cost of show route 10.20.255.2 against the expected 40. This is the lowest layer where the capstone can bite, so fix anything wrong here before looking higher.

7. **Step 7. Check the mpls.0 transit label table for a clean SID resolution.**

```
show route table mpls.0 | match 16002
```

Note: Each node-SID must resolve to a single authoritative next-hop toward its owner. From PE1 or a transit router, label 16002 must show one Swap entry heading toward PE2. If the same label resolves toward two different routers, a duplicate SID index is present and MPLS forwarding on that plane is broken. This fault only becomes cleanly observable once Step 6 has put traffic back on the correct path, so re-run this check after any underlay fix.

8. **Step 8. Confirm end-to-end tunnel resolution in inet.3.**

```
show route 10.20.255.2 table inet.3 extensive
```

Note: From PE1 the entry for PE2 must show Label operation: Push 16002 toward the correct next-hop. If inet.3 has no entry or the push is wrong, all VRF routes stay stranded in bgp.l3vpn.0 no matter how correct the route-targets are. Only when Steps 6, 7, and 8 are clean should you go back to Step 3 and judge the service layer.

Verification

show isis adjacency

```
Interface             System   L State   Hold (secs)
ge-0/0/1.0            C1       2  Up          23
ge-0/0/2.0            C2       2  Up          19
```

What to look for: Run on each router. Every expected neighbor is Up across both planes, confirming the underlay is whole and metrics are back to the design so SPF prefers the intended paths.

traceroute mpls segment-routing isis 10.20.255.2

```
Probe options: retries 3, exp 7
 ttl  Label  Protocol   Address       Previous Hop   Probe Status
   1  16002  L-ISIS     10.20.0.2     (null)        Success
 Path 1 via ge-0/0/1.0 destination 10.20.255.2
 mpls-traceroute completed
```

What to look for: From PE1 the SR label-switched path to PE2 completes end to end with label 16002. This proves the transport layer is healthy: the metric path is correct and the SID resolves to one owner with no clash.

show route table CUST_A.inet.0

```
CUST_A.inet.0: 6 destinations, 6 routes (6 active, 0 holddown)
192.168.10.0/24    *[BGP/170] 00:02:11, localpref 100
                      AS path: 65001 I, Push 16002
10.50.2.0/24       *[BGP/170] 00:02:11, localpref 100
```

What to look for: Run on PE2. The CE prefixes learned across the core are present in the CUST_A VRF, which confirms the VPN import route-target now matches target:64512:100 and the service layer has recovered on top of a clean transport.

ping 10.50.2.10 source 10.50.1.10 rapid ## from Client1 to Client2

```
PING 10.50.2.10 (10.50.2.10): 56 data bytes
!!!!!
round-trip min/avg/max = 2.1/3.4/5.0 ms
```

What to look for: The end-to-end customer test from Client1 (10.50.1.10) to Client2 (10.50.2.10) succeeds. All three layers are restored and full CUST_A service is proven from the customer edge, not just from the PE.

[screenshot 009: M8 verification: full service restored and root-cause brief delivered]

Break / Fix exercise

Scenario: The instructor has loaded the capstone fault set and customer service is down. Three faults are active at the same time, one in the IS-IS underlay, one in the SR transport, and one in the L3VPN service. From the top the symptom looks like a single service outage, but the real cause is spread across all three layers, and the lower faults are changing what the higher checks report.

Your task: Diagnose in strict layer order using the eight-step flow: prove the underlay clean, then the transport, then the service, fixing the lowest broken layer first and re-checking upward after each fix. Restore full reachability from Client1 to Client2, then deliver a five-minute root-cause brief that names each fault, states the one command output that exposed it, and explains why the diagnosis order mattered.

Hint: Resist the urge to jump straight to the service layer just because the service is what failed. A metric fault or a label clash below will mask or mimic a VRF fault above, so an empty VRF table can be lying to you. Verify each layer is genuinely clean before you move up, and re-run the checks above a layer after you fix it, because the picture changes.

The instructor holds the fault-injection and fix commands in the answer key. Diagnose it yourself first.

Brief it back

Explain each of these aloud before moving on. If you cannot, re-read the primer and re-run the verification commands.

- Brief the whole architecture layer by layer: how the IS-IS underlay feeds the SPF result, how that result drives the colored transport planes and the SID label tables, and how the L3VPN service resolves onto that transport. Make the dependency of each layer on the one below it explicit.
- Explain why diagnosing in strict layer order is the only reliable approach to a multi-fault scenario, using this capstone as the example: which lower fault

masked which higher one, and why the service-layer evidence could not be trusted until the transport under it was proven clean.

- Describe how you would communicate this root cause to a customer: what you would say the impact was, what each fault was in plain language, how you confirmed the fix, and what reassurance you would give about detection and prevention going forward.